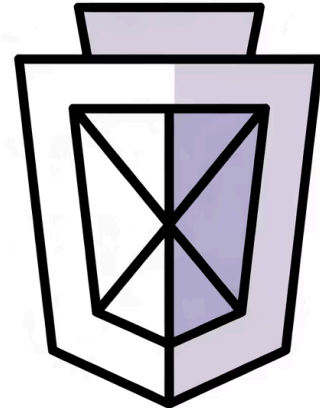


JavaScript - Jour 2

Boucles et structures de données



Objectifs pédagogiques de la journée

Répéter du code sans l'écrire plusieurs fois

Les tableaux (Arrays)

Les objets (Objects)

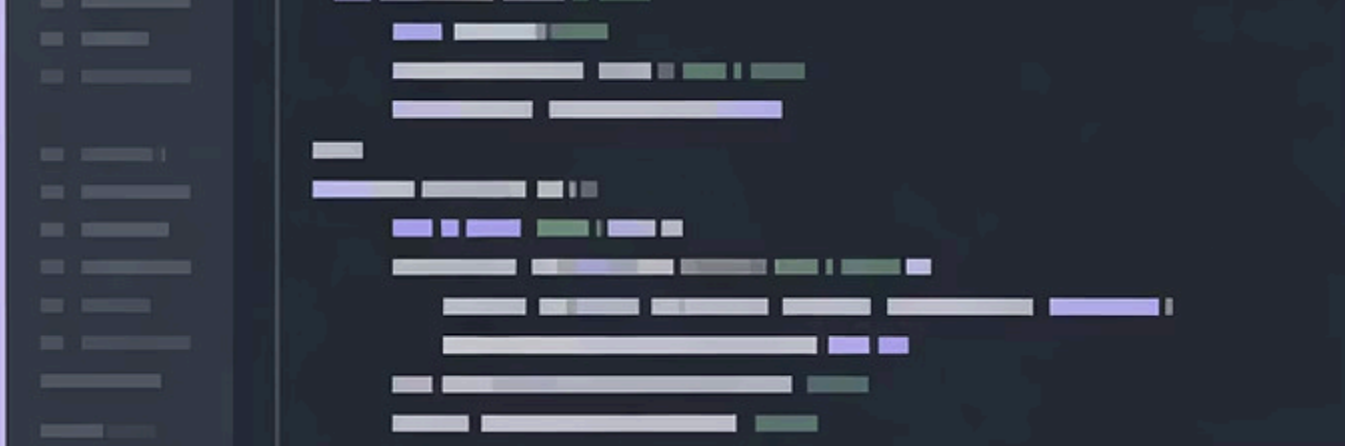
Tableaux d'objets

Les fonctions

Méthodes des tableaux

Avant de continuer...

Rendons les choses encore plus propres !



Décentraliser le JavaScript

Tout comme le CSS, il est recommandé de séparer le JavaScript du code HTML.

Étapes

1

Création du fichier JS

Créez un fichier `app.js` à la racine de votre répertoire de travail..

2

Ajout de code JS

Ajoutez n'importe quel code (pourquoi pas un exercice fait la veille !) dans le fichier JavaScript.

Un simple `console.log()` fait l'affaire.

3

Insertion du JavaScript

Modifier la balise `<script>` en supprimant son contenu. Ajoutez un attribut `src` qui pointe vers le fichier JavaScript.

```
<script src="./app.js">
```

 Ouvrez ensuite votre fichier `index.html` dans un navigateur pour vérifier si tout fonctionne !



 MODULE

Répéter du code sans l'écrire plusieurs fois

Introduction aux boucles

Les boucles permettent d'exécuter un bloc de code plusieurs fois sans avoir à le réécrire. Elles sont essentielles pour automatiser les tâches répétitives, rendant votre code plus efficace et plus concis.

Voici les trois types principaux à connaître :

1

Boucle for

Utilisée quand le nombre d'itérations est connu précisément à l'avance.

2

Boucle while

Répète un bloc de code tant qu'une condition spécifiée reste vraie.

3

Boucle do-while

Similaire à while, mais garantit que le bloc de code s'exécute au moins une fois.

La boucle for

La boucle `for` est un pilier de la programmation JavaScript, idéale lorsque vous savez précisément combien de fois une tâche doit être répétée. Elle condense en une seule ligne les mécanismes de contrôle d'itération :

```
for (initialisation; condition; incrémentation) {  
    // Code à répéter à chaque itération  
}
```

1

Initialisation

C'est ici que l'on déclare et initialise une variable de compteur (souvent `let i = 0`). Cette étape n'est exécutée qu'une seule fois, au tout début de la boucle.

2

Condition

Une expression logique (ex: `i < 10`) qui est évaluée avant chaque exécution du bloc de code. Si elle est vraie, la boucle continue ; si elle est fausse, la boucle se termine.

3

Incrémentation

Une expression (ex: `i++` ou `i += 2`) qui est exécutée après chaque itération du bloc de code. Elle sert à mettre à jour le compteur et à assurer que la condition finira par devenir fausse.

</> EXEMPLE

Exemples d'utilisations de la boucle for

```
for (let i = 1; i <= 5; i++) {  
  console.log(i); // Affiche : 1, 2, 3, 4, 5  
}
```

```
for (let i = 0; i <= 10; i += 2) {  
  console.log(i); // Affiche : 0, 2, 4, 6, 8, 10  
}
```

```
for (let i = 5; i >= 1; i--) {  
  console.log(i); // Affiche : 5, 4, 3, 2, 1  
}
```

 EXPLICATIONS

La boucle while

La boucle `while` est un autre type de boucle fondamentale en JavaScript. Elle est utilisée lorsque vous ne connaissez pas à l'avance le nombre exact d'itérations, mais que vous souhaitez répéter un bloc de code tant qu'une certaine condition reste vraie.

Syntaxe :

```
while (condition) {  
    // Code à répéter  
    // N'oubliez pas de modifier la condition à un moment  
}
```

Exemples d'utilisations de la boucle while

```
let counter = 1;

while (counter <= 5) {
  console.log(counter);
  counter++;
}
```

```
let number = 1;

while (number < 100) {
  console.log(number); // Affiche : 1, 2, 4, 8, 16, 32, 64
  number *= 2;
}
```

La boucle do-while

La boucle `do-while` est une variante de la boucle `while` qui garantit que le bloc de code est exécuté **au moins une fois**, avant même que la condition ne soit évaluée. Si la condition est vraie après la première exécution, la boucle continue comme une boucle `while` classique.

Syntaxe :

```
do {  
    // Code à exécuter  
    // S'assure que ce code est exécuté au moins une fois  
} while (condition); // La condition est vérifiée APRÈS la première exécution
```

Exécution Garantie

Le bloc de code s'exécute toujours au moins une fois, même si la condition est fausse dès le départ.

Vérification Tardive

La condition est évaluée après la première itération, contrôlant les itérations suivantes.

</> EXEMPLE

Exemple d'utilisation de la boucle do-while

```
let choice;

do {
  console.log("=== Menu ===");
  console.log("1. Option 1");
  console.log("2. Option 2");
  console.log("3. Quitter");
  choice = 2; // Simulation d'un choix utilisateur
  console.log(`Vous avez choisi : ${choice}`);
} while (choice !== 3);

console.log("Au revoir !");
```

24078	1	≡	+	×
19587	8	=	0	=

 MISE EN PRATIQUE

Exercice guidé : Table de multiplication

Objectif : Afficher la table de multiplication de 7 en utilisant une boucle `for`.

Corrigé : Table de multiplication

```
const number = 7;

console.log(`Table de multiplication de ${number} :`);

for (let i = 1; i <= 10; i++) {
  const result = number * i;
  console.log(`${number} × ${i} = ${result}`);
}
```

Sortie :

```
Table de multiplication de 7 :
7 × 1 = 7
7 × 2 = 14
7 × 3 = 21
7 × 4 = 28
7 × 5 = 35
7 × 6 = 42
7 × 7 = 49
7 × 8 = 56
7 × 9 = 63
7 × 10 = 70
```



 MISE EN PRATIQUE

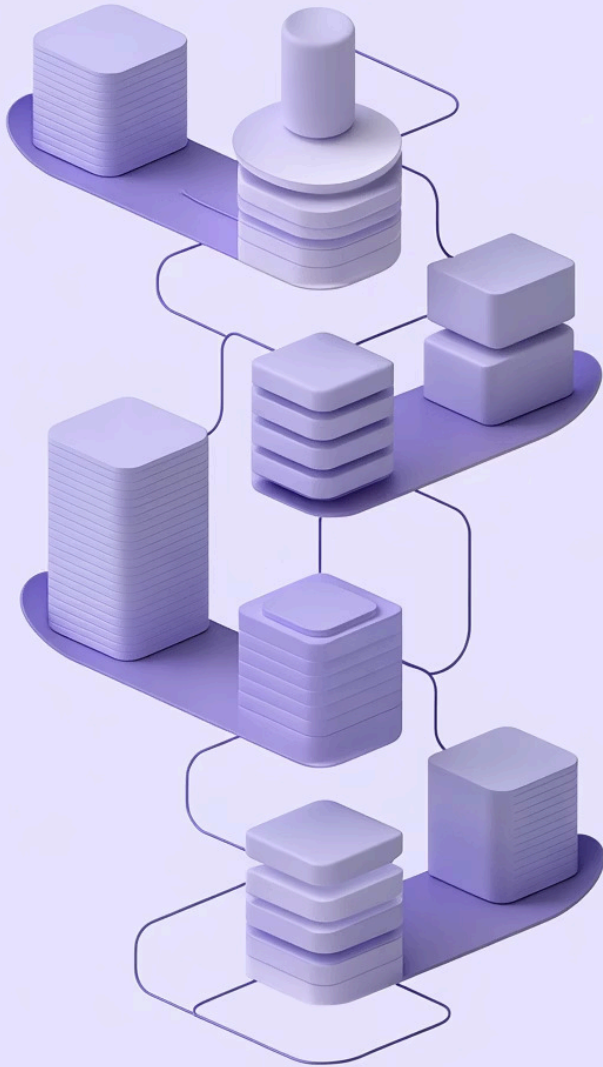
Calculateur de somme

Écrire un programme qui calcule la somme de tous les nombres de 1 à X.

Résultat attendu, pour $X = 100$:

La somme de 1 à 100 est : 5050

 **Bonus pour les plus rapides :**
Additionner uniquement les nombres premiers



 MODULE

Les tableaux (Arrays)

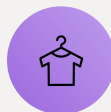
Introduction aux tableaux

Un tableau est une structure de données qui permet de stocker et d'organiser plusieurs valeurs en un endroit. C'est l'un des outils les plus couramment utilisés en JavaScript pour gérer des collections de données, comme une liste de noms, une série de nombres ou un ensemble d'objets.



Structure de données

Un moyen organisé de conserver et manipuler des ensembles d'informations de manière cohérente.



Stocke plusieurs valeurs

Peut contenir divers types de données : nombres, textes, objets, fonctions, ou d'autres tableaux.



Une variable unique

Toutes les valeurs sont regroupées et accessibles via un nom de variable commun.

Création et syntaxe des tableaux

En JavaScript, la manière la plus courante de créer un tableau est d'utiliser la syntaxe des littéraux de tableau, c'est-à-dire des crochets `[]`.

Syntaxe :

```
const tableau = [valeur1, valeur2, valeur3, ...];
```

Les crochets `[]`

Ils encadrent la liste des éléments et signalent à JavaScript que vous créez un nouveau tableau.

Éléments séparés par des virgules

Chaque valeur (nombre, chaîne, objet, etc.) est un élément distinct du tableau et doit être séparée par une virgule.

Types de données mixtes

Un tableau JavaScript peut contenir des éléments de différents types de données au sein du même tableau.

</> EXEMPLE

Exemples concrets de tableaux

```
const firstnames = ["Alice", "Bob", "Charlie"]; // Un tableau de chaînes de caractères (strings)
const numbers = [10, 20, 30, 40, 50];          // Un tableau de nombres
const mixed = ["texte", 42, true, null];       // Un tableau de types de données mélangés
```

⚠ Attention tout de même à **éviter les tableaux mixtes** ! Conventionnellement, on préférera avoir un seul et même type dans tout le tableau.

Accéder aux éléments d'un tableau

Les éléments d'un tableau sont organisés séquentiellement et sont accessibles via un **index numérique**.

Il est important de se souvenir qu'en JavaScript (comme dans la plupart des langages de programmation), le premier élément d'un tableau a toujours l'index **0**.

```
const fruits = ["Pomme", "Banane", "Orange", "Fraise"];
```

```
console.log(fruits[0]); // "Pomme"
```

```
console.log(fruits[1]); // "Banane"
```

```
console.log(fruits[3]); // "Fraise"
```

```
console.log(fruits[10]); // undefined (n'existe pas)
```

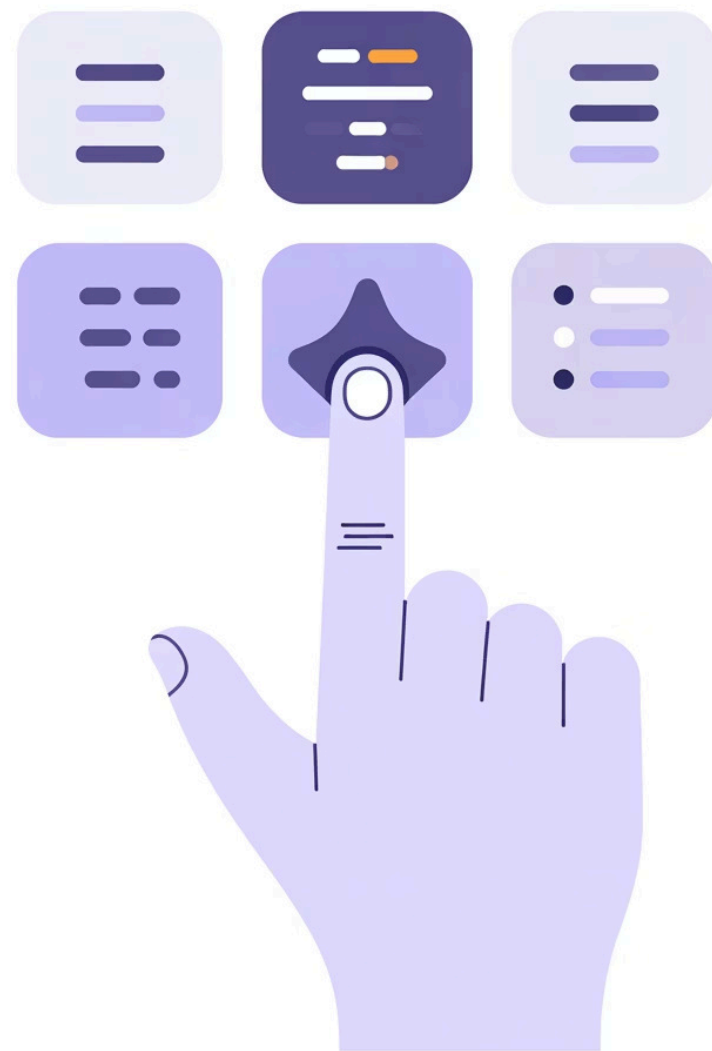
EXPLICATIONS

Modifier un élément d'un tableau

Vous pouvez modifier la valeur d'un élément existant dans un tableau en utilisant son index. Il suffit d'assigner une nouvelle valeur à l'emplacement souhaité.

```
fruits[1] = "Mangue";  
console.log(fruits);  
// Affiche : ["Pomme", "Mangue", "Orange", "Fraise"]
```

Dans cet exemple, l'élément à l'index 1 (qui était "Banane") est remplacé par "Mangue".



La propriété `length`

La propriété `length` retourne le nombre d'éléments actuellement présents dans le tableau.

```
const numbers = [5, 10, 15, 20];  
console.log(numbers.length);           // Affiche : 4  
  
// Accéder au dernier élément du tableau  
console.log(numbers[numbers.length - 1]); // Affiche : 20
```

Compte les éléments

Retourne un nombre entier représentant la quantité d'éléments dans le tableau.

Base 1, pas 0

Contrairement aux index qui commencent à 0, la propriété `length` commence à compter à partir de 1.

Accès dynamique

Permet d'accéder au dernier élément d'un tableau, même si sa taille est inconnue ou variable.

Méthodes de base pour manipuler les tableaux



`push()` - Ajouter à la fin

Ajoute un ou plusieurs éléments à la fin du tableau et retourne la nouvelle longueur du tableau.



`pop()` - Retirer de la fin

Supprime le dernier élément du tableau et le retourne. Cette méthode modifie la longueur du tableau.



`unshift()` - Ajouter au début

Ajoute un ou plusieurs éléments au début du tableau et retourne la nouvelle longueur. Les éléments existants sont décalés vers la droite.



`shift()` - Retirer du début

Supprime le premier élément du tableau et le retourne. Tous les autres éléments sont décalés vers la gauche.

</> EXEMPLE

La méthode push()

```
const animals = ["Chat", "Chien"];  
animals.push("Oiseau");           // Ajoute "Oiseau" à la fin du tableau  
  
console.log(animals);             // ["Chat", "Chien", "Oiseau"]
```

La méthode pop()

```
const animals = ["Chat", "Chien", "Oiseau"];  
const lastAnimal = animals.pop(); // Retourne le dernier élément et le supprime du tableau  
  
console.log(lastAnimal);          // "Oiseau"  
console.log(animals);            // ["Chat", "Chien"]
```

</> EXEMPLE

La méthode unshift()

```
const numbers = [2, 3, 4];  
numbers.unshift(1);           // Ajoute 1 au début du tableau  
  
console.log(numbers);         // [1, 2, 3, 4]
```

La méthode shift()

```
const numbers = [1, 2, 3, 4];  
const firstNumber = numbers.shift(); // Retourne le premier élément et le supprime du tableau  
  
console.log(firstNumber);       // 1  
console.log(numbers);           // [2, 3, 4]
```

</> EXEMPLE

Parcourir un tableau avec `for`

Méthode classique :

```
const grades = [12, 15, 18, 10, 14];

for (let i = 0; i < grades.length; i++) {
  console.log(`Note ${i + 1} : ${grades[i]}/20`);
}
```

Sortie :

```
Note 1 : 12/20
Note 2 : 15/20
Note 3 : 18/20
Note 4 : 10/20
Note 5 : 14/20
```

 EXPLICATIONS

Parcourir un tableau avec for...of

La boucle `for...of` est une manière moderne de parcourir les éléments d'un tableau. Elle permet d'accéder directement à la valeur de chaque élément, sans avoir à gérer l'index.

Syntaxe :

```
for (const element of tableau) {  
    // Code à exécuter pour chaque élément  
}
```

</> EXEMPLE

Exemple de for...of

```
const colors = ["Rouge", "Vert", "Bleu", "Jaune"];

for (const color of colors) {
  console.log(`Couleur : ${color}`);
}
```

Sortie :

```
Couleur : Rouge
Couleur : Vert
Couleur : Bleu
Couleur : Jaune
```

Quand utiliser for vs for...of ?

1

Avec for (boucle classique)

Utilisez cette boucle lorsque vous avez besoin d'accéder à l'**index** de chaque élément en plus de sa valeur.



Avec for...of

Préférez `for...of` lorsque vous avez besoin d'itérer directement sur les **valeurs** des éléments du tableau, sans vous soucier de leur index.

Exercice guidé : Calcul de moyenne

Objectif : Calculer la moyenne des notes d'un tableau.

Nous allons partir d'un tableau de notes prédéfini et d'une variable pour accumuler la somme de ces notes, avant de calculer la moyenne finale.

```
const grades = [12, 15, 18, 10, 14, 16];  
let gradesSum = 0;
```

</> EXEMPLE

Corrigé : Calcul de moyenne

```
// Calcul de la somme des notes
for (const grade of grades) {
  gradesSum += grade;
}

// Calcul de la moyenne et affichage
const average = gradesSum / grades.length;
console.log(`Moyenne : ${average}/20`);
```



Gestionnaire de températures

Consigne

Créer un programme qui :

1. Stocke les températures de la semaine dans un tableau (7 valeurs).
2. Affiche chaque température avec le jour correspondant.
3. Calcule et affiche la température moyenne de la semaine.
4. Trouve et affiche la température la plus haute.
5. Trouve et affiche la température la plus basse.

Sortie attendue

Températures de la semaine :

Lundi : 18°C

Mardi : 20°C

...

Température moyenne : 19.5°C

Température maximale : 23°C

Température minimale : 15°C

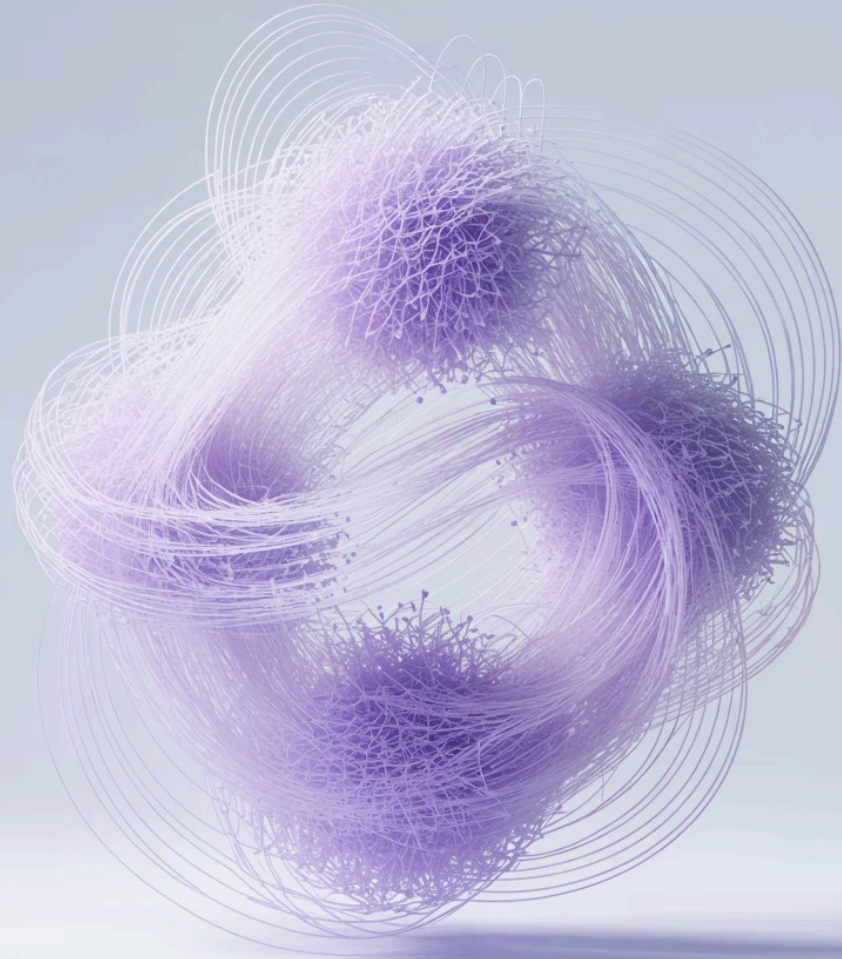


Bonus pour les plus rapides

Compter et afficher combien de jours ont eu une température supérieure à la moyenne.

 MODULE

Les objets (Objects)





EXPLICATIONS

Introduction aux objets

Un objet permet de regrouper des données liées entre elles sous forme de propriétés. Chaque propriété est une paire clé-valeur, où la clé (ou nom de la propriété) est une chaîne de caractères (ou un symbole), et la valeur peut être n'importe quel type de donnée (un nombre, une chaîne, un tableau, une fonction, ou même un autre objet).

Syntaxe de création d'objets

Les objets sont créés en utilisant des accolades `{}` pour encapsuler une liste de paires clé-valeur, séparées par des virgules. C'est la manière la plus courante et directe de définir un objet en JavaScript.

```
const monObjet = {  
  cle1: valeur1,  
  propriete2: "texte",  
  nombreDeVies: 3,  
  estActif: true,  
};
```

Décorticage de la syntaxe de création d'objets

Les accolades {}

Elles définissent les limites de l'objet. Tout ce qui se trouve à l'intérieur constitue les propriétés de l'objet.

Les clés (Propriétés)

Ce sont les identifiants uniques de chaque élément de donnée. Elles sont généralement des chaînes de caractères mais n'ont pas besoin d'être entre guillemets si ce sont des noms de variable valides.

Les valeurs

Chaque clé est associée à une valeur. Cette valeur peut être de n'importe quel type : chaîne, nombre, booléen, tableau, fonction, ou même un autre objet.

Deux-points et virgules

Le deux-points : sépare une clé de sa valeur (clé: valeur), et les virgules , séparent les différentes paires clé-valeur.

Accès aux propriétés d'un objet

Pour interagir avec les données stockées dans un objet (lire, modifier ou ajouter des propriétés), JavaScript offre deux syntaxes distinctes.

Notation pointée (recommandée)

Cette méthode est la plus courante et la plus simple. Elle utilise un point `.` suivi du nom de la propriété.

```
const personne = {  
  prenom: "Alice",  
  age: 25,  
};  
  
console.log(personne.prenom); // "Alice"  
console.log(personne.age);   // 25
```

Notation entre crochets

Cette syntaxe utilise des crochets `[]` avec le nom de la propriété sous forme de chaîne de caractères (ou de variable).

```
console.log(personne["prenom"]); // "Alice"  
console.log(personne["age"]);   // 25  
  
// Utile avec des variables  
const propriete = "prenom";  
console.log(personne[propriete]); // "Alice"
```

Modifier et ajouter des propriétés

```
const car = {  
  brand: "Peugeot",  
  model: "208",  
  year: 2020,  
};  
  
// Modifier une propriété existante  
car.year = 2021;  
  
// Ajouter une nouvelle propriété  
car.color = "Rouge";  
  
console.log(car);  
// { brand: "Peugeot", model: "208", year: 2021, color: "Rouge" }
```

</> EXEMPLE

Supprimer une propriété

```
const user = {  
  name: "Dupont",  
  email: "dupont@email.com",  
  password: "secret123",  
};  
  
delete user.password;  
  
console.log(user);  
// { name: "Dupont", email: "dupont@email.com" }
```

Objets complexes (propriétés imbriquées)

Un objet peut contenir d'autres objets ou des tableaux comme valeurs de ses propriétés.

```
const student = {  
  name: "Martin",  
  age: 22,  
  adress: {  
    street: "10 rue de la Paix",  
    city: "Lyon",  
    zipCode: "69000",  
  },  
  grades: [15, 18, 12, 16],  
};  
  
console.log(student.adress.city); // Affiche "Lyon"  
console.log(student.grades[0]);  // Affiche 15
```

Méthodes Object utiles

JavaScript offre plusieurs méthodes intégrées pour interagir avec les objets, permettant d'extraire leurs clés, leurs valeurs ou même leurs paires clé-valeur sous forme de tableaux.



Object.keys()

Retourne un tableau contenant les noms (clés) de toutes les propriétés énumérables propres à un objet.



Object.values()

Retourne un tableau contenant les valeurs de toutes les propriétés énumérables propres à un objet.



Object.entries()

Retourne un tableau de paires [clé, valeur] pour toutes les propriétés énumérables propres à un objet.

 EXPLICATIONS

Parcourir un objet avec for...in

La boucle `for...in` est spécialement conçue pour itérer sur les propriétés énumérables d'un objet.

```
for (const key in object) {  
  console.log(key);      // Nom de la propriété  
  console.log(object[key]); // Valeur de la propriété  
}
```

Exemple d'itération avec for...in

```
const product = {  
  name: "Ordinateur portable",  
  price: 899,  
  brand: "Dell",  
  inStock: true,  
};  
  
for (const property in product) {  
  console.log(`${property} : ${product[property]}`);  
}
```

Sortie

```
name : Ordinateur portable  
price : 899  
brand : Dell  
inStock : true
```

Fiche utilisateur

Objectif : Créer et afficher une fiche utilisateur complète en utilisant un objet et une boucle `for...in` pour parcourir ses propriétés.

Sortie attendue

```
=== Fiche Utilisateur ===  
firstname : Sophie  
name : Bernard  
age : 28  
email : sophie.bernard@email.com  
city : Marseille  
job : Développeuse
```

Bonus pour les plus rapides

Créer une collection d'utilisateurs puis afficher les fiches de tous les utilisateurs.

MODULE

Tableaux d'objets

Style			Nono	7
				21.9Z
				15.00
				15.37
				20.42
				15.4Z
				16.63
				23.22
				15.55

Introduction aux tableaux d'objets

Avec JavaScript, il est possible de stocker et organiser plusieurs objets en un seul endroit : **dans un tableau**.

Le fonctionnement de la collection est naturellement celui des tableaux, tandis que les valeurs (objets) conservent leur fonctionnement bien à eux.

Exemple d'une liste d'étudiants

```
const students = [  
  {  
    lastname: "Dupont",  
    firstname: "Alice",  
    age: 20,  
    average: 15,  
  },  
  {  
    lastname: "Martin",  
    firstname: "Bob",  
    age: 22,  
    average: 18,  
  },  
  {  
    lastname: "Bernard",  
    firstname: "Charlie",  
    age: 21,  
    average: 12,  
  }  
];
```

Chaque élément du tableau est un objet distinct.

</> EXEMPLE

Accéder aux propriétés

```
// Premier étudiant (l'objet complet)
console.log(students[0]);
// { lastname: "Dupont", firstname: "Alice", age: 20, average: 15 }

// Prénom du deuxième étudiant
console.log(students[1].firstname);
// "Bob"

// Moyenne du troisième étudiant
console.log(students[2].average);
// 12
```

</> EXEMPLE

Parcourir un tableau d'objets

```
const products = [  
  { name: "Clavier", price: 45, inStock: true },  
  { name: "Souris", price: 25, inStock: false },  
  { name: "Écran", price: 250, inStock: true },  
];  
  
for (const product of products) {  
  console.log(`${product.name} - ${product.price}€`);  
}
```



MISE EN PRATIQUE

Exercice guidé : Calcul de la moyenne des prix des articles en stock

Objectif : À partir du tableau d'articles fourni ci-dessous, calculer et afficher la moyenne des prix de tous les articles dont la propriété `inStock` vaut `true`.

Tableau d'articles de départ

```
const products = [  
  { name: "Clavier", price: 45, inStock: true },  
  { name: "Souris", price: 25, inStock: false },  
  { name: "Écran", price: 250, inStock: true },  
];
```

</> EXEMPLE

Corrigé : Calcul de la moyenne des prix

```
let priceSum = 0;
let inStockProductsCount = 0;

for (const product of products) {
  if (product.inStock) {
    priceSum += product.price;
    inStockProductsCount++;
  }
}

const priceAverage = priceSum / inStockProductsCount;
console.log(`Moyenne des prix des articles disponibles : ${priceAverage}`);
```

Exercice : Système de gestion de notes avancé

Créez un tableau JavaScript contenant au moins 5 objets étudiants. Chaque objet étudiant doit avoir les propriétés suivantes :

- `lastname` (chaîne de caractères)
- `firstname` (chaîne de caractères)
- `grades` (un tableau de 4 nombres pour les notes)

Votre programme doit :

1. Calculer la moyenne de chaque étudiant.
2. Afficher chaque étudiant avec sa moyenne.
3. Déterminer l'étudiant avec la meilleure moyenne.
4. Déterminer l'étudiant avec la moins bonne moyenne.
5. Calculer la moyenne générale de la classe.

Bonus : Système de gestion de notes avancé

Bonus pour les plus rapides

Ajoutez une mention pour chaque étudiant basée sur sa moyenne :

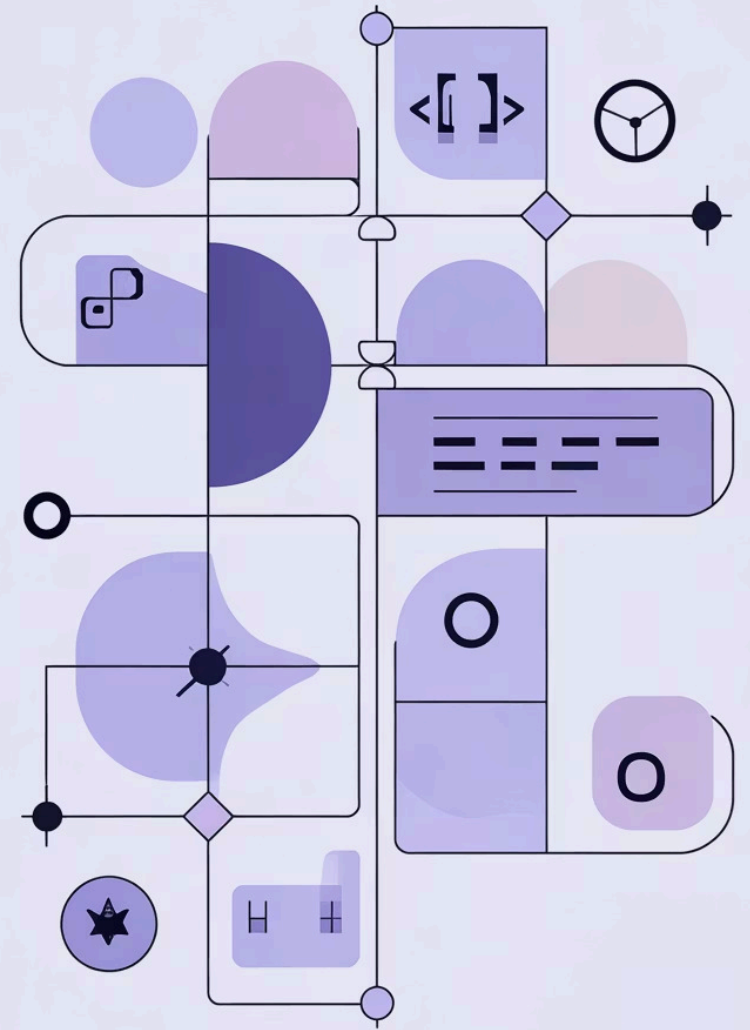
- **Très bien** : moyenne ≥ 16
- **Bien** : moyenne ≥ 14
- **Assez bien** : moyenne ≥ 12
- **Passable** : moyenne ≥ 10
- **Insuffisant** : moyenne < 10

Bonus de la mort

Allez plus loin en calculant la médiane de la classe et en identifiant les étudiants au-dessus et en-dessous de la moyenne générale.

MODULE

Les fonctions



The background features a light purple and white illustration. It includes several overlapping windows or code editors, some containing stylized code snippets like curly braces and lines of text. There are also icons: a person with arms raised in a square, a code editor icon, and a question mark in a square. These elements are connected by thin, light-colored lines, suggesting a network or flow of information.

EXPLICATIONS

Qu'est-ce qu'une fonction ?

Les fonctions permettent de regrouper du code réutilisable, d'organiser un programme et d'éviter les répétitions. Elles sont des blocs de code autonomes conçus pour effectuer une tâche spécifique, rendant votre code plus **modulaire** et **facile à maintenir**.

Déclaration de fonction

Une **déclaration de fonction** (ou *function statement*) définit une fonction nommée. C'est la manière la plus courante de créer une fonction.

```
function maFonction(parametre1, parametre2) {  
  // Code à exécuter  
  return resultat;  
}
```



EXEMPLE

Exemple de fonction basique

```
function add(a, b) {  
  return a + b;  
}
```

```
const sumResult = add(1, 2); // 3
```

Paramètres par défaut des fonctions

En JavaScript, il est possible d'assigner une **valeur par défaut** à un paramètre de fonction. Cette valeur sera utilisée si aucun argument n'est fourni pour ce paramètre lors de l'appel de la fonction, ou si `undefined` est explicitement passé.

```
function greet(firstName, greeting = "Bonjour") {  
  console.log(`${greeting} ${firstName} !`);  
}  
  
// Appel avec un seul argument (greeting utilise sa valeur par défaut)  
greet("Alice"); // "Bonjour Alice !"  
  
// Appel avec deux arguments (greeting est écrasé)  
greet("Bob", "Salut"); // "Salut Bob !"  
  
greet("Marcel", undefined); // "Bonjour Marcel !"
```

Le mot-clé return

Le mot-clé `return` est fondamental pour les fonctions. Il a deux rôles principaux : il met fin à l'exécution de la fonction et renvoie une valeur spécifique au code qui l'a appelée. Dès qu'un `return` est rencontré, la fonction s'arrête immédiatement et le reste de son code n'est pas exécuté.

```
// Return arrête l'exécution de la fonction
function checkAge(age) {
  if (age < 18) {
    return "Mineur"; // La fonction s'arrête ici si l'âge est inférieur à 18
  }
  // Si age >= 18, le code continue jusqu'ici
  return "Majeur"; // La fonction retourne "Majeur" et s'arrête
}

console.log(checkAge(15)); // Affiche "Mineur"
console.log(checkAge(20)); // Affiche "Majeur"
```

Les fonctions fléchées

Les fonctions fléchées (*Arrow Functions*) offrent une syntaxe plus courte pour écrire des expressions de fonction.

```
// Syntaxe basique
const maFonction = (param1, param2) => {
  // Corps de la fonction
  return param1 + param2;
};

// Avec un seul paramètre (les parenthèses sont optionnelles, mais recommandées)
const carre = (nombre) => nombre * nombre;

// Sans paramètre
const direBonjour = () => "Bonjour !";

// Retour implicite pour les fonctions à une seule expression
const addition = (a, b) => a + b;
```



MISE EN PRATIQUE

Exercice guidé : Récupérer la valeur maximale

Objectif : Créer une fonction JavaScript qui prend un tableau de nombres en argument et renvoie la valeur maximale trouvée dans ce tableau. Il **n'est pas autorisé** d'utiliser la méthode intégrée `Math.max()` (ce serait trop facile).

Exemple de tableau de départ

```
const nombres = [15, 8, 23, 4, 42, 10];  
// La fonction doit retourner 42 pour ce tableau.
```

Corrigé : Récupérer la valeur maximale

```
function getMaxNumber(numbers) {  
  if (numbers.length === 0) {  
    return undefined;  
  }  
  
  let maxNumber = numbers[0];  
  for (let i = 1; i < numbers.length; i++) {  
    if (numbers[i] > maxNumber) {  
      maxNumber = numbers[i];  
    }  
  }  
  
  return maxNumber;  
}  
  
const numbers = [15, 8, 23, 4, 42, 10];  
const max = getMaxNumber(numbers);  
console.log(`Le nombre maximal est : ${max}`);
```

Calcul du prix Toutes Taxes Comprises (TTC)

Objectif : Créez une fonction JavaScript qui prend en entrée un prix Hors Taxes (HT) et un taux de TVA (en pourcentage), puis retourne le prix Toutes Taxes Comprises (TTC).

Fonction à implémenter :

```
function calculatePriceTTC(priceHT, vatRate) {  
  // Votre code ici  
  // Formule : priceHT * (1 + vatRate / 100)  
}  
  
// Exemples d'utilisation :  
console.log(calculatePriceTTC(100, 20)); // Devrait afficher 120  
console.log(calculatePriceTTC(50, 5.5)); // Devrait afficher 52.75
```

Calcul du prix Toutes Taxes Comprises (TTC)

Bonus : Fonctions complémentaires

Pour aller plus loin, créez les fonctions suivantes pour un système de calcul de commande avancé :

- `calculateDiscount(price, discountPercentage)` : Une fonction qui applique une réduction (en pourcentage) à un prix donné et retourne le prix réduit.
- `calculateShippingFees(weight)` : Calcule les frais de livraison en fonction du poids (en kg) de l'article :
 - Moins de 1 kg : 5€
 - Entre 1 kg et moins de 5 kg : 10€
 - 5 kg ou plus : 15€
- `calculateFinalTotal(priceHT, vatRate, discountPercentage, weight)` : Une fonction qui combine toutes les fonctions précédentes pour calculer le prix final d'un article après application de la TVA, d'une réduction et des frais de port.

MODULE

Méthodes de tableaux



Méthodes de tableaux

1 Maîtriser `map()`

Pour transformer des données de manière efficace.

2 Utiliser `filter()`

Pour sélectionner des éléments spécifiques selon des critères.

3 Comprendre `reduce()`

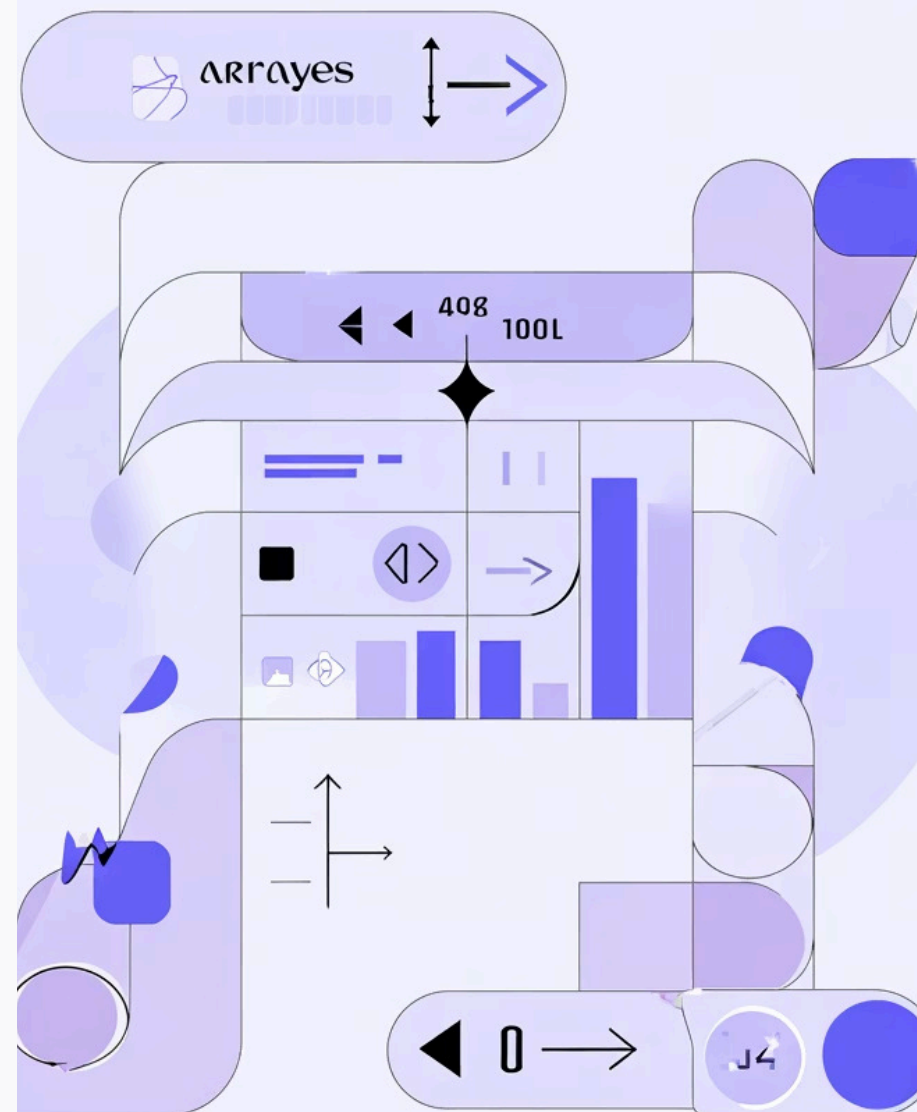
Pour agréger et accumuler des valeurs.

4 Exploiter `find()`

Pour rechercher et récupérer le premier élément correspondant.

5 Trier avec `sort()`

Pour organiser les éléments d'un tableau dans un ordre précis.



map() : Transformer chaque élément

La méthode `map()` permet de créer un **nouveau tableau** en appelant une fonction fournie sur chaque élément du tableau d'origine. Elle ne modifie pas le tableau initial, mais retourne un nouveau tableau avec les résultats de l'appel de la fonction sur chaque élément.

```
const products = [  
  { name: "Laptop", price: 1000 },  
  { name: "Souris", price: 25 },  
  { name: "Clavier", price: 75 },  
];  
  
const names = products.map((product) => product.name);  
console.log(names); // ["Laptop", "Souris", "Clavier"]
```

filter() : Filtrer des éléments

La méthode `filter()` permet de créer un **nouveau tableau** contenant tous les éléments du tableau d'origine qui satisfont à une condition donnée. Le callback fourni à `filter()` doit retourner une valeur booléenne :

- `true` pour inclure l'élément dans le nouveau tableau
- `false` pour l'exclure.

Comme `map()`, cette méthode ne modifie pas le tableau initial.

```
const people = [  
  { name: "Alice", age: 25 },  
  { name: "Bob", age: 17 },  
  { name: "Charlie", age: 30 },  
  { name: "Diana", age: 16 }  
];  
  
const adults = people.filter((person) => person.age >= 18);  
console.log(adults); // Un tableau contenant les objets d'Alice et Charlie
```

reduce() : Accumuler des valeurs

La méthode `reduce()` applique une fonction (appelée "réducteur") sur chaque élément d'un tableau afin de le réduire à une **seule valeur**. Elle itère sur le tableau de gauche à droite, accumulant une seule valeur de retour.

```
const numbers = [1, 2, 3, 4, 5];

const sum = numbers.reduce(
  (accumulator, number) => {
    return accumulator + number;
  },
  0 // 0 est la valeur initiale
);

console.log(sum); // 15
```

EXPLICATIONS

find() : Trouver un élément

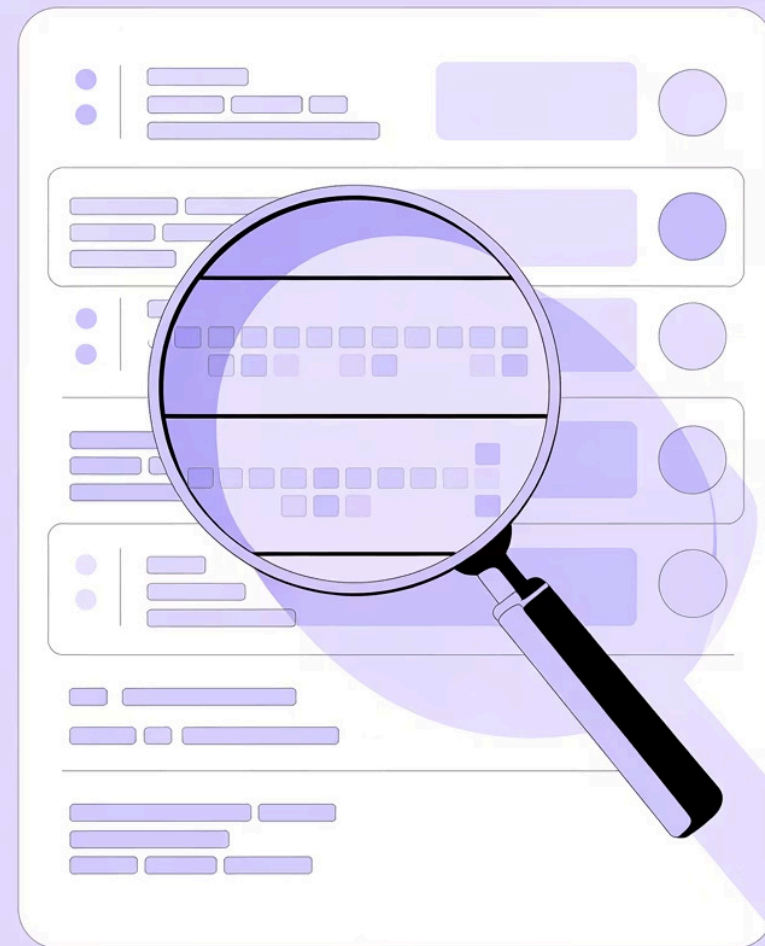
La méthode `find()` est utilisée pour récupérer la **valeur du premier élément** dans le tableau fourni qui satisfait la fonction de test fournie. Si aucun élément ne satisfait la fonction de test, `undefined` est renvoyé. Cette méthode ne modifie pas le tableau d'origine.

EXEMPLE

```
const numbers = [5, 12, 8, 130, 44];

// Trouver le premier nombre > 10
const foundNumber = numbers.find(
  (number) => number > 10
);

console.log(foundNumber); // 12
```



EXPLICATIONS

sort() : Trier un tableau

La méthode `sort()` trie les éléments d'un tableau directement dans le tableau d'origine (attention : elle modifie le tableau initial !). Elle renvoie une référence au tableau trié. Par défaut, le tri est effectué en comparant les éléments comme des chaînes de caractères, ce qui peut donner des résultats inattendus pour les nombres.

```
const numbers = [40, 1, 5, 200];  
  
numbers.sort();  
console.log(numbers);
```

D'après vous, quelle est la valeur de `numbers` après l'exécution du code ?

sort() : Le tri numérique

Par défaut, `sort()` effectue un tri lexicographique. Pour trier des nombres correctement, vous devez fournir une fonction de comparaison.

```
const numbers = [40, 1, 5, 200];  
  
// Tri numérique croissant  
numbers.sort((a, b) => a - b);  
console.log(numbers); // [1, 5, 40, 200]
```

La fonction `(a, b) => a - b` retourne un nombre négatif si `a` est plus petit que `b`, positif si `a` est plus grand, et zéro si égaux. C'est le comportement attendu pour un tri numérique croissant.

EXPLICATIONS

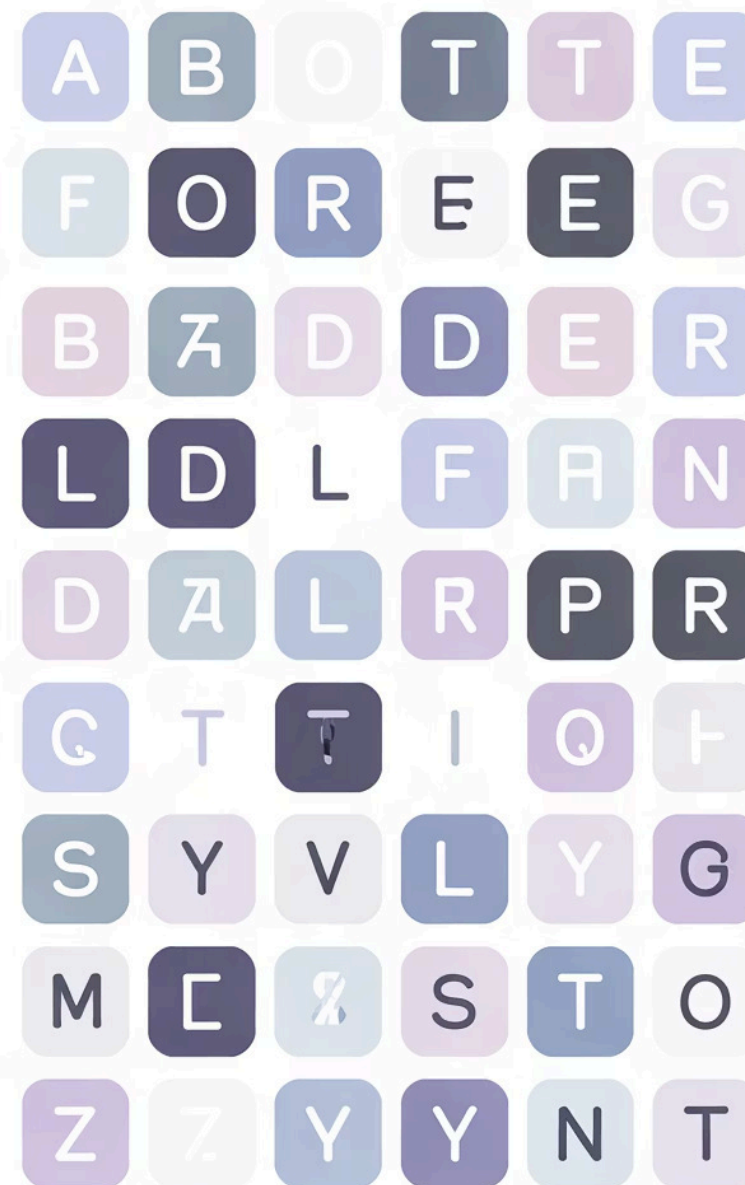
sort() : Trier des chaînes de caractères

Lorsque vous triez des chaînes de caractères, la méthode `sort()` par défaut peut donner des résultats inattendus avec des caractères spéciaux ou des majuscules/minuscules.

Pour un tri correct, sensible à la langue et aux conventions locales, utilisez la méthode `localeCompare()` dans votre fonction de comparaison. Elle compare deux chaînes de caractères et retourne un nombre indiquant si la première chaîne vient avant, après ou est la même que la seconde.

```
const firstnames = ["Chloé", "Lilwenn", "Soline", "Gabin", "Louise"];

firstnames.sort((a, b) => a.localeCompare(b));
console.log(firstnames);
// ["Chloé", "Gabin", "Lilwenn", "Louise", "Soline"]
```



Exercice guidé : Manipulation de tableaux

Nous allons utiliser le jeu de données `orders` pour effectuer plusieurs manipulations de tableaux courantes en JavaScript.

01

Calculer le montant total avec frais

Pour chaque commande, calculez le montant total incluant des frais de service de 10%.

02

Filtrer les commandes livrées

Filtrez les commandes pour ne garder que celles dont le statut est "livrée".

03

Calculer le chiffre d'affaires total

Calculez le chiffre d'affaires total généré par toutes les commandes.

04

Trouver une commande spécifique

Trouvez la commande ayant l'identifiant 3.

05

Trier les commandes par montant

Triez les commandes par leur montant, du plus élevé au plus bas.

Données initiales : Manipulation de tableaux

```
const orders = [  
  { id: 1, customer: "Alice", amount: 150, status: "livrée", products: 3 },  
  { id: 2, customer: "Bob", amount: 85, status: "en cours", products: 2 },  
  { id: 3, customer: "Charlie", amount: 200, status: "livrée", products: 5 },  
  { id: 4, customer: "Diana", amount: 45, status: "annulée", products: 1 },  
  { id: 5, customer: "Alice", amount: 120, status: "livrée", products: 2 }  
];
```

Corrigé : Exercice guidé de manipulation de tableaux

```
// 1. map() : Ajouter 10% de frais de service
const withFees = orders.map((order) => ({
  ...order,
  totalAmount: order.amount * 1.10
}));
console.log("Commandes avec frais :", withFees);

// 2. filter() : Garder uniquement les commandes livrées
const delivered = orders.filter((order) => order.status === "livrée");
console.log("Commandes livrées :", delivered);

// 3. reduce() : Calculer le chiffre d'affaires total
const revenue = orders.reduce((acc, order) => acc + order.amount, 0);
console.log("Chiffre d'affaires total :", revenue);

// 4. find() : Trouver une commande spécifique
const foundOrder = orders.find((order) => order.id === 3);
console.log("Commande trouvée (ID 3) :", foundOrder);

// 5. sort() : Trier par montant décroissant
const sorted = [...orders].sort((a, b) => b.amount - a.amount);
console.log("Commandes triées par montant :", sorted);
```

Exercice guidé : Analyse de données de ventes

Utilisez le tableau `sales` pour effectuer les manipulations de données suivantes, en appliquant les méthodes de tableaux JavaScript que nous venons de voir.

```
const sales = [
  { product: "Laptop", quantity: 2, unitPrice: 899, category: "Informatique" },
  { product: "Souris", quantity: 15, unitPrice: 25, category: "Accessoires" },
  { product: "Clavier", quantity: 8, unitPrice: 75, category: "Accessoires" },
  { product: "Écran", quantity: 3, unitPrice: 299, category: "Informatique" },
  { product: "Webcam", quantity: 0, unitPrice: 89, category: "Accessoires" },
  { product: "Casque", quantity: 12, unitPrice: 45, category: "Audio" }
];
```

01

Calculer le chiffre d'affaires de chaque vente

Ajoutez une propriété `totalSale` à chaque objet de vente, représentant le prix total (quantité * prix unitaire).

02

Filtrer les ventes actives

Ne conservez que les ventes dont la quantité est supérieure à 0.

03

Calculer le chiffre d'affaires total global

Sommez les `totalSale` de toutes les ventes pour obtenir le chiffre d'affaires global.

04

Trouver les détails d'un produit

Recherchez et affichez l'objet de vente intégrant la lettre "c" (non sensible à la casse, "C" inclus).

05

Trier les ventes par chiffre d'affaires décroissant

Organisez le tableau des ventes actives du plus grand au plus petit `totalSale`.

Chaînage de méthodes

Le chaînage de méthodes en JavaScript permet d'appliquer plusieurs transformations successives sur un tableau ou un objet en une seule expression, rendant le code plus concis et souvent plus lisible. Chaque méthode retourne une nouvelle collection (ou une valeur), sur laquelle la méthode suivante peut être appliquée.

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

// Sans chaînage (approche étape par étape)
const evenNumbers = numbers.filter((number) => number % 2 === 0); // [2, 4, 6, 8, 10]
const doubles = evenNumbers.map((number) => number * 2);           // [4, 8, 12, 16, 20]
const sum = doubles.reduce((acc, number) => acc + number, 0);      // 60
console.log("Résultat sans chaînage :", sum);

// Avec chaînage (une seule expression)
const result = numbers
  .filter((number) => number % 2 === 0)           // Filtre les nombres pairs
  .map((number) => number * 2)                   // Double chaque nombre
  .reduce((acc, number) => acc + number, 0);      // Calcule la somme

console.log("Résultat avec chaînage :", result); // 60
```

CONCLUSION

Et... on s'arrête là pour les notions du jour !

Ça fait beaucoup (vraiment), donc on fait une pause pour aujourd'hui.

... mais ce n'est pas tout à fait fini 😊



Projet : Système de gestion de notes d'école

Nous allons mettre en pratique toutes les notions de JavaScript abordées aujourd'hui en développant un système complet de gestion des notes pour une école. Ce projet vous permettra de manipuler des structures de données complexes et d'appliquer les méthodes de tableau vues précédemment.

Fonctionnalités attendues

1 Gestion des étudiants

Définir une structure de données pour stocker les informations des étudiants, y compris leurs identifiants, noms, classes et un tableau de leurs matières avec les notes associées.

2 Calculer la moyenne par matière

Une fonction pour calculer la moyenne d'un étudiant pour une matière donnée à partir de son tableau de notes.

3 Calculer la moyenne générale

Une fonction pour déterminer la moyenne générale d'un étudiant en se basant sur toutes ses matières.

4 Rechercher un étudiant

Trouver un étudiant spécifique par son identifiant unique.

5 Filtrer par classe

Obtenir une liste de tous les étudiants appartenant à une classe spécifique.

6 Top des moyennes

Identifier et lister les meilleurs étudiants en fonction de leur moyenne générale.

7 Statistiques par classe

Calculer des statistiques (moyenne, médiane, écart-type) pour une classe donnée.

8 Statistiques par matière

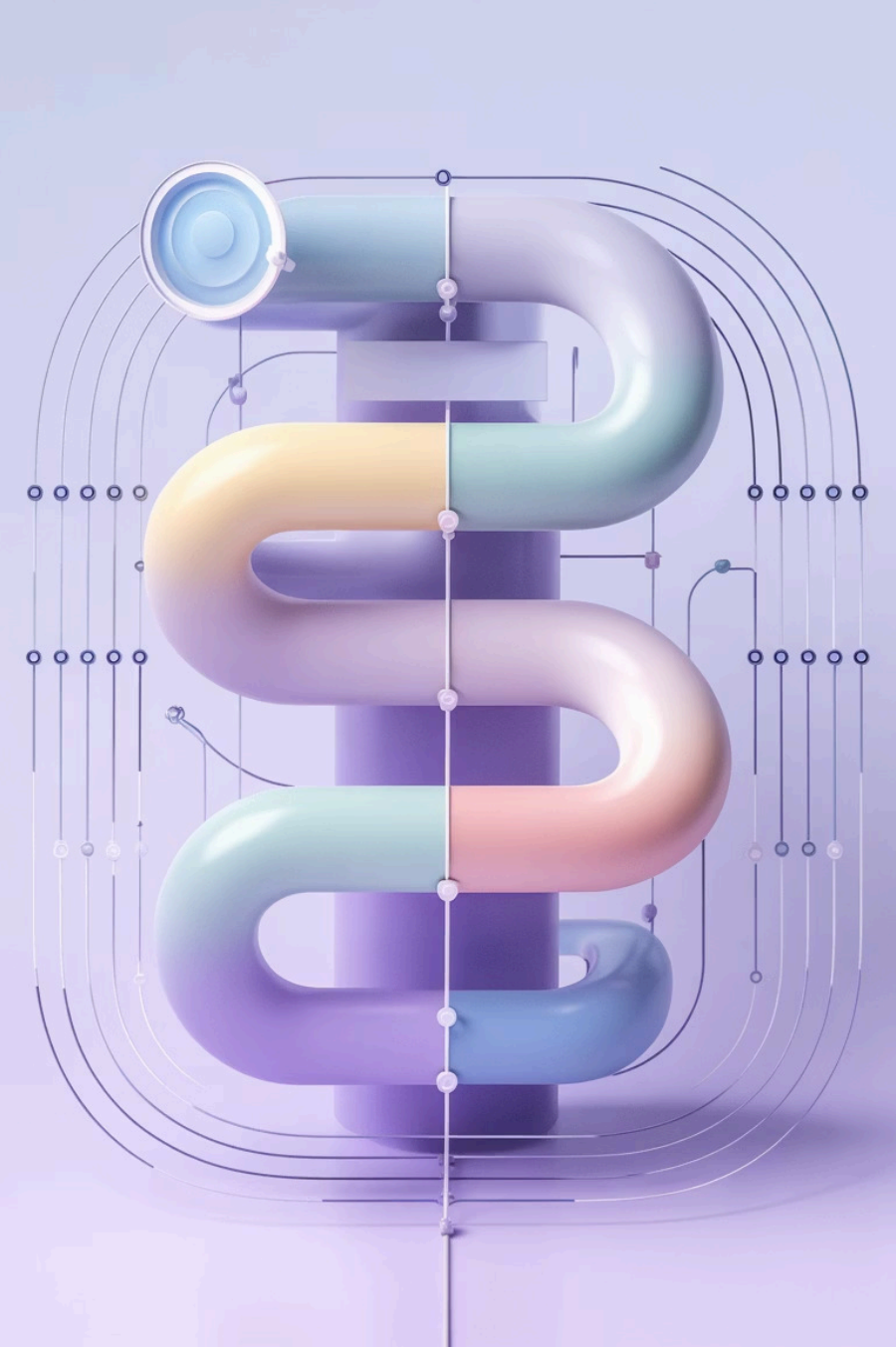
Analyser les performances globales pour une matière spécifique.

9 Générer un bulletin

Créer un rapport détaillé pour un étudiant, incluant ses moyennes par matière et sa moyenne générale.

10 Générer un classement

Produire un classement général des étudiants basé sur leur moyenne.



▶▶ À SUIVRE

Prochaines étapes

Demain, nous allons enfin pouvoir faire du vrai JavaScript web, en découvrant le DOM.

Et comme d'habitude, ce qu'on a vu jusqu'à aujourd'hui sera approfondi au fur et à mesure !